

DISCRETE-TIME SIGNAL GENERATION AND MANIPULATION IN MATLAB

Objective

The objectives of this experiment are:

- To generate and visualize discrete-time unit impulse and unit step signals in MATLAB.
- To develop reusable MATLAB functions such as `impseq`, `stepseq`, `sigadd`, `sigshift`, `sigfold`, `sigmult`, and `energypower` for signal generation and manipulation.
- To perform fundamental discrete-time signal operations including addition, scaling, shifting, folding, and multiplication.
- To construct complex discrete-time sequences by combining elementary signals and applying signal transformations.
- To study the effect of windowing on discrete sinusoidal signals and observe changes in signal behavior.
- To compute and analyze the energy and average power of discrete-time signals.
- To interpret stem plots and relate graphical representations to their mathematical definitions.
- To verify analytical results through MATLAB simulation and develop debugging skills for signal-processing applications.
- To gain practical understanding of fundamental concepts used in Digital Signal Processing (DSP).

Theory

Introduction

Discrete-time signals are signals defined only at discrete instants of time and are represented using an integer index n . These signals are fundamental in Digital Signal Processing (DSP), where real-world analog signals are sampled and processed in discrete form.

MATLAB provides a convenient environment for generating, manipulating, and visual-

izing discrete-time signals. In this experiment, elementary signals such as unit impulse and unit step sequences are generated and combined to create more complex signals. Signal operations including shifting, folding, addition, and multiplication are also performed. In addition, the energy and average power of discrete-time signals are computed to analyze their characteristics.

Discrete-Time Unit Impulse Signal

The discrete-time unit impulse signal, also known as the Dirac delta sequence, is defined as:

$$\delta[n] = \begin{cases} 1, & n = 0 \\ 0, & n \neq 0 \end{cases} \quad (1)$$

A shifted impulse sequence is represented as:

$$\delta[n - n_0] \quad (2)$$

where n_0 denotes the shift amount.

Discrete-Time Unit Step Signal

The discrete-time unit step sequence is defined as:

$$u[n] = \begin{cases} 1, & n \geq 0 \\ 0, & n < 0 \end{cases} \quad (3)$$

A delayed step sequence is represented as:

$$u[n - n_0] \quad (4)$$

where n_0 indicates the delay.

Signal Operations

Several basic operations can be performed on discrete-time signals.

Signal Shifting

A shifted signal is represented as:

$$y(n) = x(n - n_0) \quad (5)$$

A positive value of n_0 delays the signal, while a negative value advances it.

Signal Folding

Time reversal or folding is given by:

$$y(n) = x(-n) \quad (6)$$

This operation reflects the signal about the vertical axis.

Signal Addition

Two discrete signals can be added as:

$$y(n) = x_1(n) + x_2(n) \quad (7)$$

Signal Multiplication

Point-wise multiplication is given by:

$$y(n) = x_1(n)x_2(n) \quad (8)$$

Windowing of Signals

Windowing is the process of restricting a signal to a specific interval. A window function multiplies the original signal and forces values outside the desired interval to become zero.

Windowing is commonly used in spectral analysis, filtering, and signal segmentation.

Energy and Power of Discrete-Time Signals

The energy of a discrete-time signal is defined as:

$$E = \sum_{n=-\infty}^{\infty} |x(n)|^2 \quad (9)$$

The average power of a signal over N samples is:

$$P = \frac{1}{N} \sum_{n=0}^{N-1} |x(n)|^2 \quad (10)$$

Energy and power calculations help characterize the strength of a signal.

Applications of Discrete-Time Signal Processing

Discrete-time signal processing is widely used in:

- Digital communication systems
- Audio and speech processing
- Image and video processing
- Biomedical signal analysis
- Radar and sonar systems
- Control and automation systems

Understanding discrete-time signals and their operations is essential for advanced Digital Signal Processing applications.

MATLAB Codes

Helper Function: `impseq.m`

```
function [x, n] = impseq(n0, n1, n2)
% Generates  $x(n) = \delta(n - n_0)$  over  $n_1 \leq n \leq n_2$ 
% Usage: [x, n] = impseq(n0, n1, n2)

n = n1:n2;
x = [(n - n0) == 0];

end
```

Task 1: Basic Impulse Signals

```
clc;
close all;
clear all;

% delta[n]
[delta1,n] = impseq(0,-5,10);

% delta[n-4]
[delta2,n] = impseq(4,-5,10);

subplot(1,2,1)
stem(n,delta1,'filled');
```

```

xlabel('n');
ylabel('amplitude');
title('unit impulse signal \delta[n] / Himal Joshi, 28');
grid on;

subplot(1,2,2)
stem(n,delta2,'filled');
xlabel('n');
ylabel('amplitude');
title('shifted impulse signal \delta[n-4] / Himal Joshi, 28');
grid on;

```

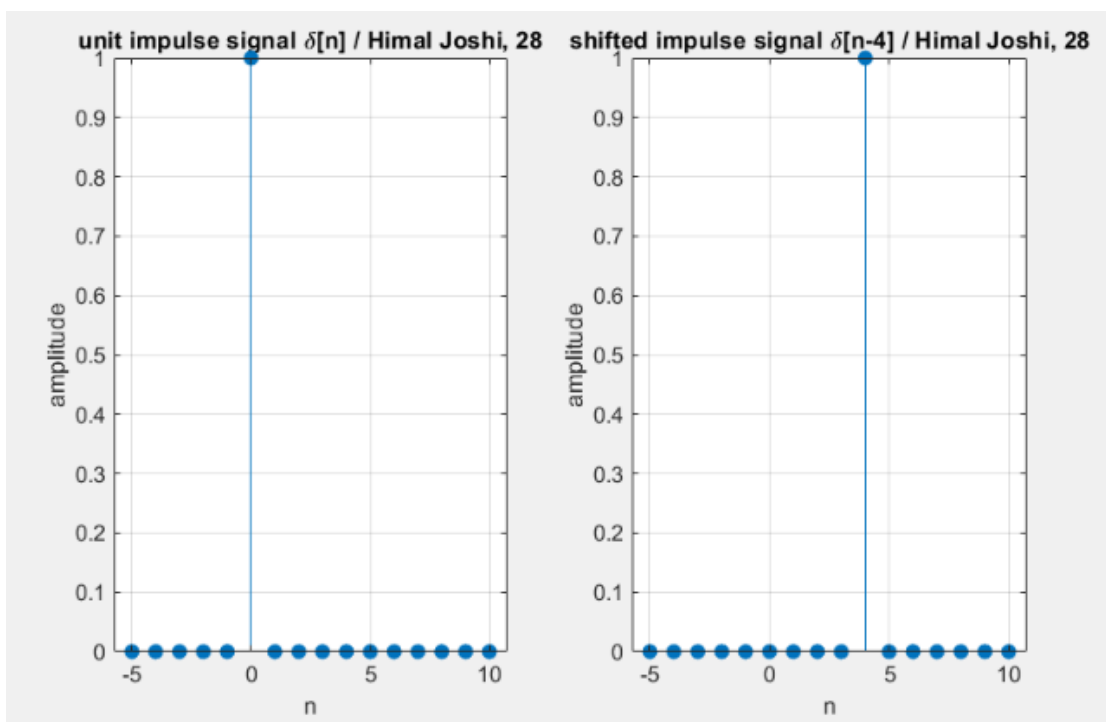


Figure 1: Basic Impulse Signals

Task 2: Scaled Impulse Combination

```

clc;
close all;
clear all;

% Generate impulses
[d1,n] = impseq(-2,-5,5); % delta(n+2)
[d2,n] = impseq(4,-5,5);  % delta(n-4)

% Composite signal

```

```

x = 2*d1 - d2;

stem(n,x,'filled');
hold on;

% Highlight non-zero samples
stem(-2,2,'r','filled');
stem(4,-1,'r','filled');

text(-2,2.2,'(-2, 2)');
text(4,-0.8,'(4, -1)');

xlabel('n');
ylabel('amplitude');
title('x(n)=2\delta(n+2)-\delta(n-4) / Himal Joshi, 28');
grid on;
hold off;

```

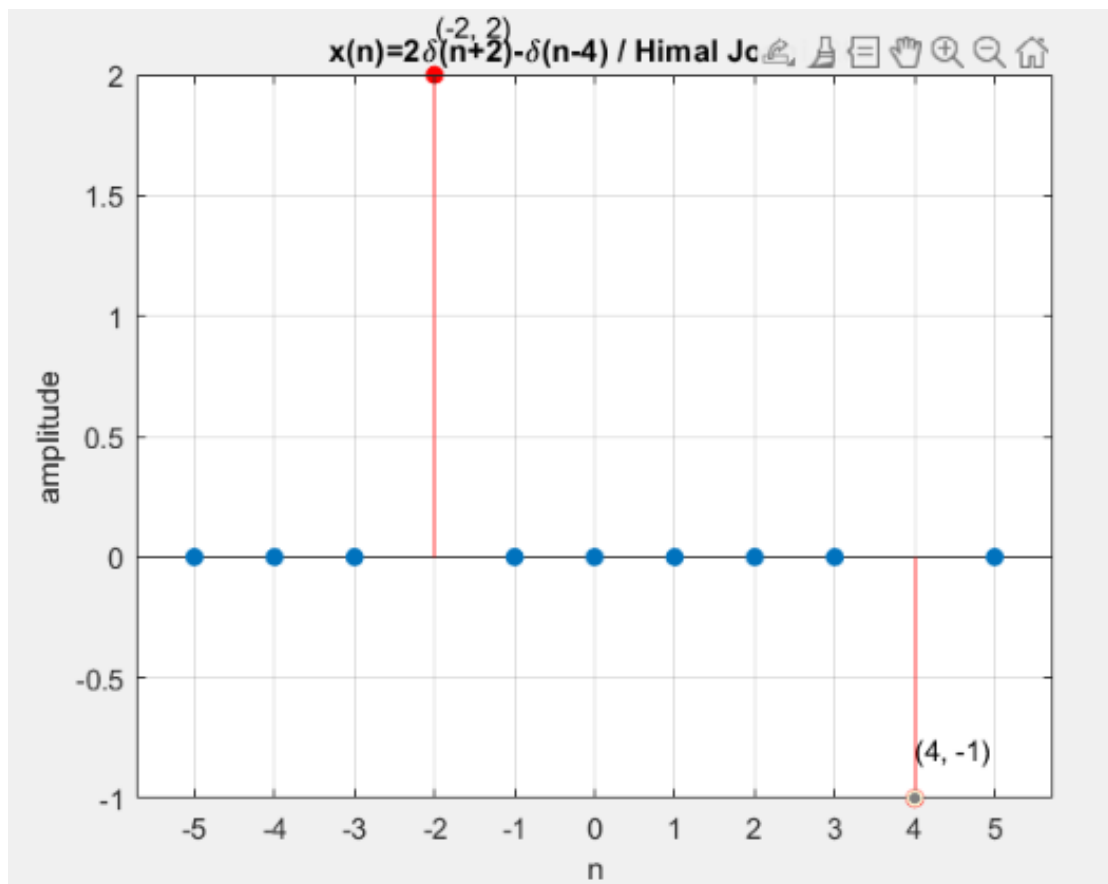


Figure 2: Scaled Impulse Combination

Helper Function: stepseq.m

```
function [x, n] = stepseq(n0, n1, n2)
% Generates  $x(n) = u(n - n0)$  over  $n1 \leq n \leq n2$ 
% Usage: [x, n] = stepseq(n0, n1, n2)

n = n1:n2;
x = [(n - n0) >= 0];

end
```

Task 3: Step Signal Plots

```
clc;
close all;
clear all;

% u[n]
[u1,n] = stepseq(0,-5,10);

% u[n-3]
[u2,n] = stepseq(3,-5,10);

subplot(1,2,1)
stem(n,u1,'filled');
xlabel('n');
ylabel('amplitude');
title('unit step signal u[n] / Himal Joshi, 28');
grid on;

subplot(1,2,2)
stem(n,u2,'filled');
xlabel('n');
ylabel('amplitude');
title('delayed step signal u[n-3] / Himal Joshi, 28');
grid on;
```

Task 4: Piecewise Exponential-Linear Sequence

```
clc;
close all;
clear all;
```

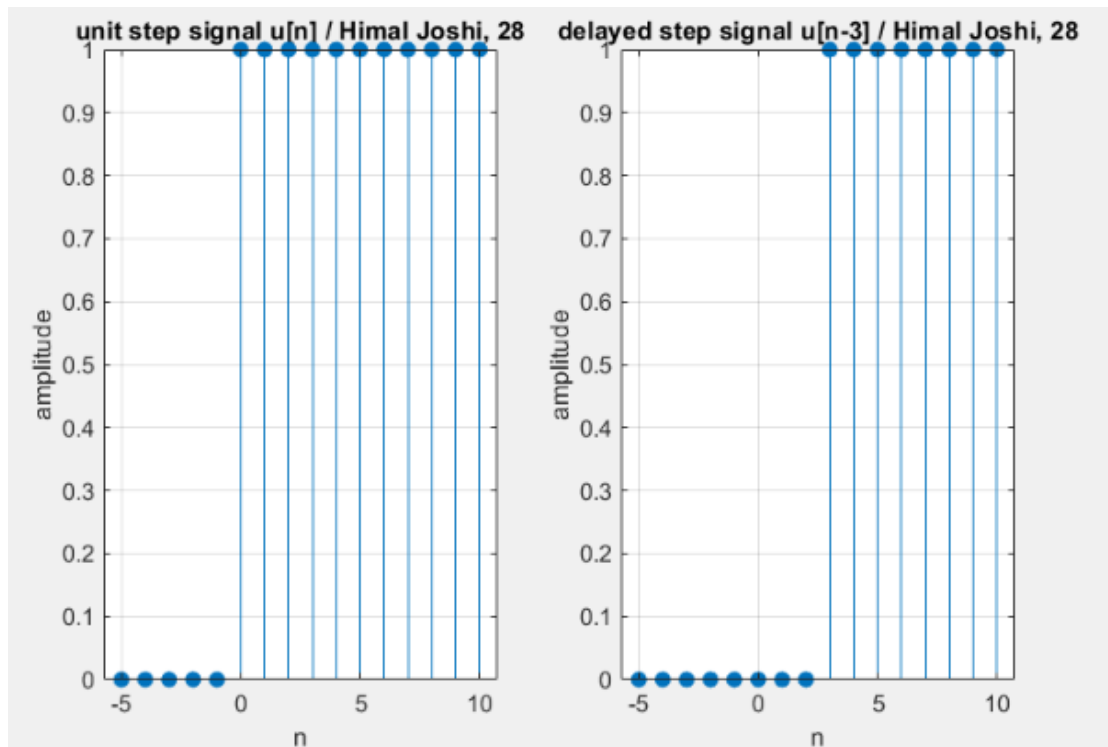


Figure 3: Step Signal Plots

```

n = 0:20;

[u1,~] = stepseq(0,0,20);
[u2,~] = stepseq(10,0,20);
[u3,~] = stepseq(20,0,20);

% Linear rise region
x1 = n .* (u1 - u2);

% Exponential decay region
x2 = 10*exp(-0.3*(n-10)) .* (u2 - u3);

% Composite signal
x = x1 + x2;

stem(n,x,'filled');
hold on;

xline(10,'--r','n=10');

```



```

text(3,6,'Linear rise');
text(12,7,'Exponential decay');

xlabel('n');
ylabel('amplitude');
title('Piecewise Exponential-Linear Sequence / Himal Joshi, 28');
grid on;

hold off;

```

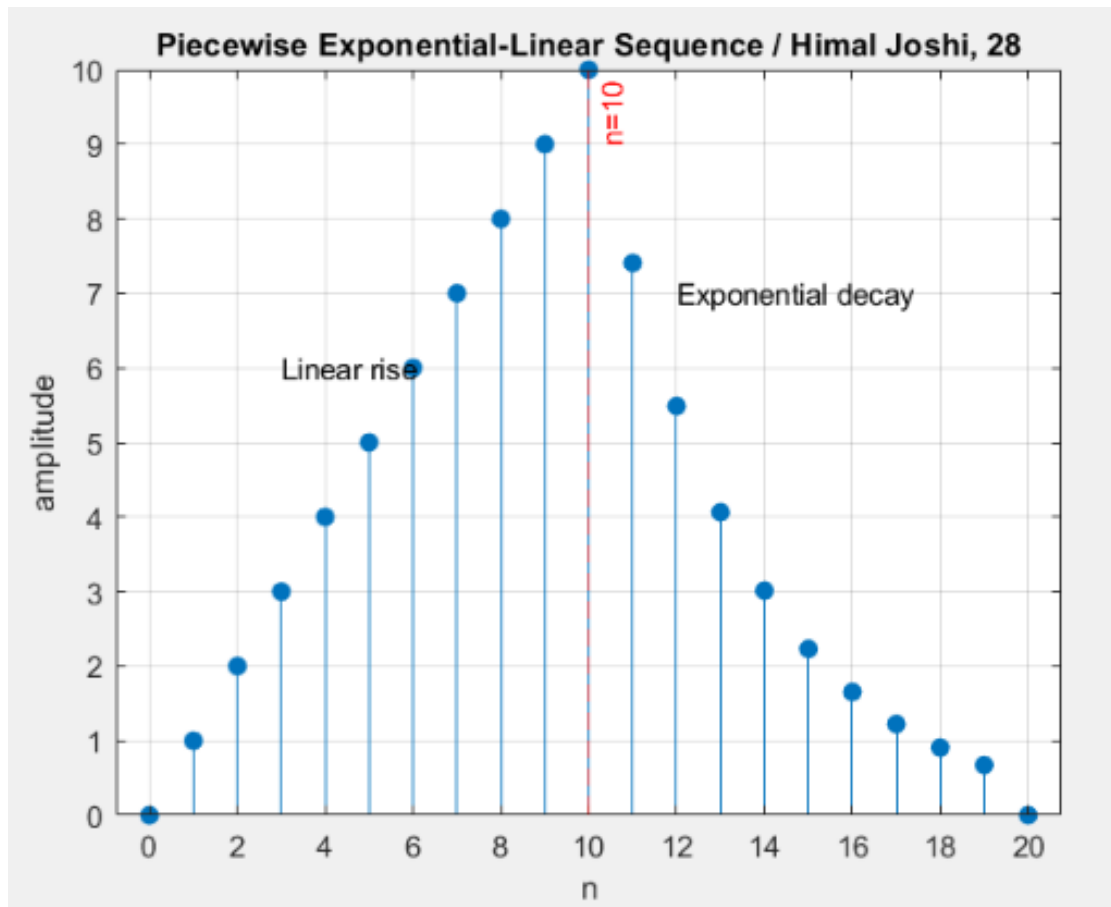


Figure 4: Piecewise Exponential-Linear Sequence

Helper Function: sigadd.m

```

function [y, ny] = sigadd(x1, n1, x2, n2)

% General addition y(n) = x1(n) + x2(n)

ny = min(min(n1),min(n2)) : max(max(n1),max(n2));

y1 = zeros(1,length(ny));

```

```

y2 = zeros(1,length(ny));

y1((ny >= min(n1)) & (ny <= max(n1))) = x1;
y2((ny >= min(n2)) & (ny <= max(n2))) = x2;

y = y1 + y2;

end

```

Helper Function: sigshift.m

```

function [y, ny] = sigshift(x, n, n0)

% Shift:  $y(n) = x(n - n_0)$ 

y = x;
ny = n + n0;

end

```

Helper Function: sigfold.m

```

function [y, n] = sigfold(x, n)

% Fold:  $y(n) = x(-n)$ 

y = fliplr(x);
n = -fliplr(n);

end

```

Helper Function: sigmult.m

```

function [y, ny] = sigmult(x1, n1, x2, n2)

% Element-wise multiplication

ny = min(min(n1),min(n2)) : max(max(n1),max(n2));

y1 = zeros(1,length(ny));
y2 = zeros(1,length(ny));

```

```

y1((ny >= min(n1)) & (ny <= max(n1))) = x1;
y2((ny >= min(n2)) & (ny <= max(n2))) = x2;

y = y1 .* y2;

end

```

Task 5: Shifting and Combination

```

clc;
close all;
clear all;

n = -2:10;
x = [1:7,6:-1:1];

% 2x(n-5)
[xs1,ns1] = sigshift(x,n,5);
xs1 = 2*xs1;

% -3x(n+4)
[xs2,ns2] = sigshift(x,n,-4);
xs2 = -3*xs2;

% x1(n)
[x1,n1] = sigadd(xs1,ns1,xs2,ns2);

subplot(3,1,1)
stem(ns1,xs1,'filled');
xlabel('n');
ylabel('amplitude');
title('2x(n-5) / Himal Joshi, 28');
grid on;

subplot(3,1,2)
stem(ns2,xs2,'filled');
xlabel('n');
ylabel('amplitude');
title('-3x(n+4) / Himal Joshi, 28');
grid on;

```

```

subplot(3,1,3)
stem(n1,x1,'filled');
xlabel('n');
ylabel('amplitude');
title('x_1(n)=2x(n-5)-3x(n+4) / Himal Joshi, 28');
grid on;

```

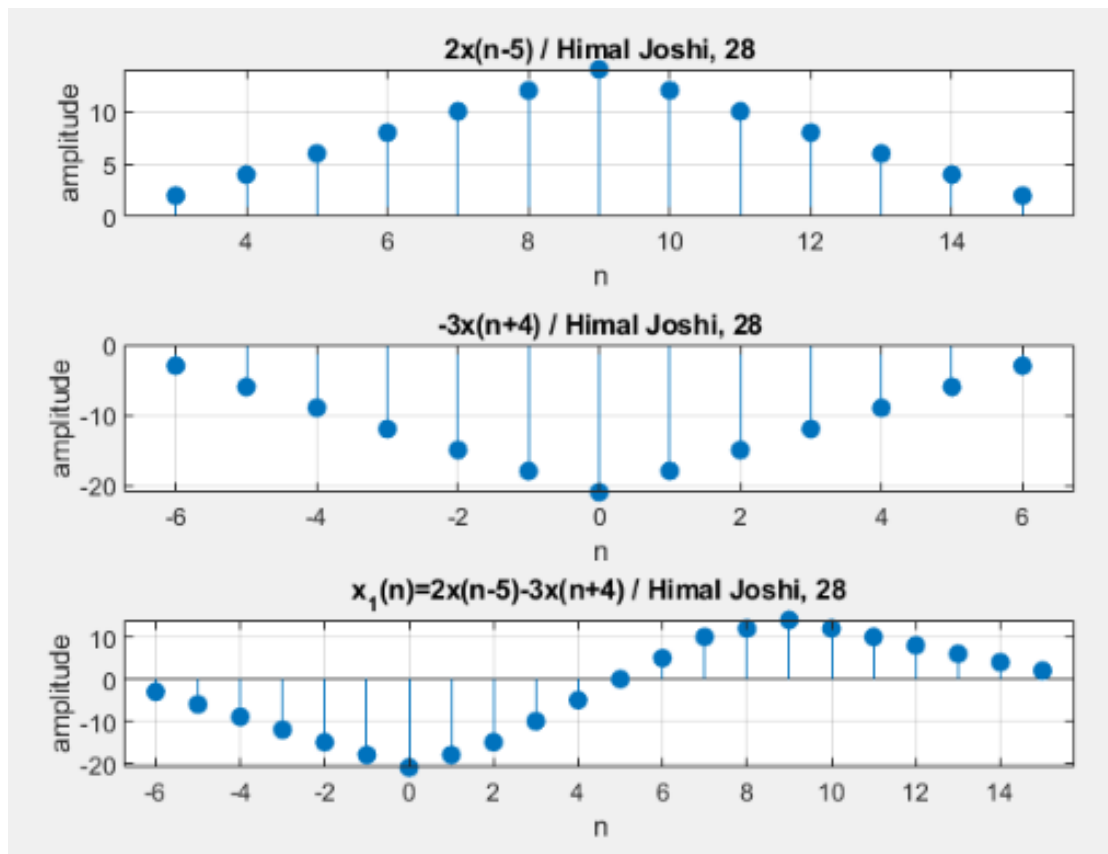


Figure 5: Shifting and Combination

Task 6: Folding and Multiplication

```

clc;
close all;
clear all;

n = -2:10;
x = [1:7,6:-1:1];

% x(3-n) = fold then shift by 3
[xf,nf] = sigfold(x,n);
[xfs,nfs] = sigshift(xf,nf,3);

```

```

% x(n-2)
[xs,ns] = sigshift(x,n,2);

% x(n)x(n-2)
[xm,nm] = sigmult(x,n,xs,ns);

% x2(n)=x(3-n)+x(n)x(n-2)
[x2,n2] = sigadd(xfs,nfs,xm,nm);

subplot(3,1,1)
stem(nfs,xfs,'filled');
xlabel('n');
ylabel('amplitude');
title('x(3-n) / Himal Joshi, 28');
grid on;

subplot(3,1,2)
stem(nm,xm,'filled');
xlabel('n');
ylabel('amplitude');
title('x(n)x(n-2) / Himal Joshi, 28');
grid on;

subplot(3,1,3)
stem(n2,x2,'filled');
xlabel('n');
ylabel('amplitude');
title('x_2(n)=x(3-n)+x(n)x(n-2) / Himal Joshi, 28');
grid on;

```

Task 7: Windowed Sinusoidal Signal

```

clc;
close all;
clear all;

n = 0:40;

[u1,~] = stepseq(0,0,40);

```

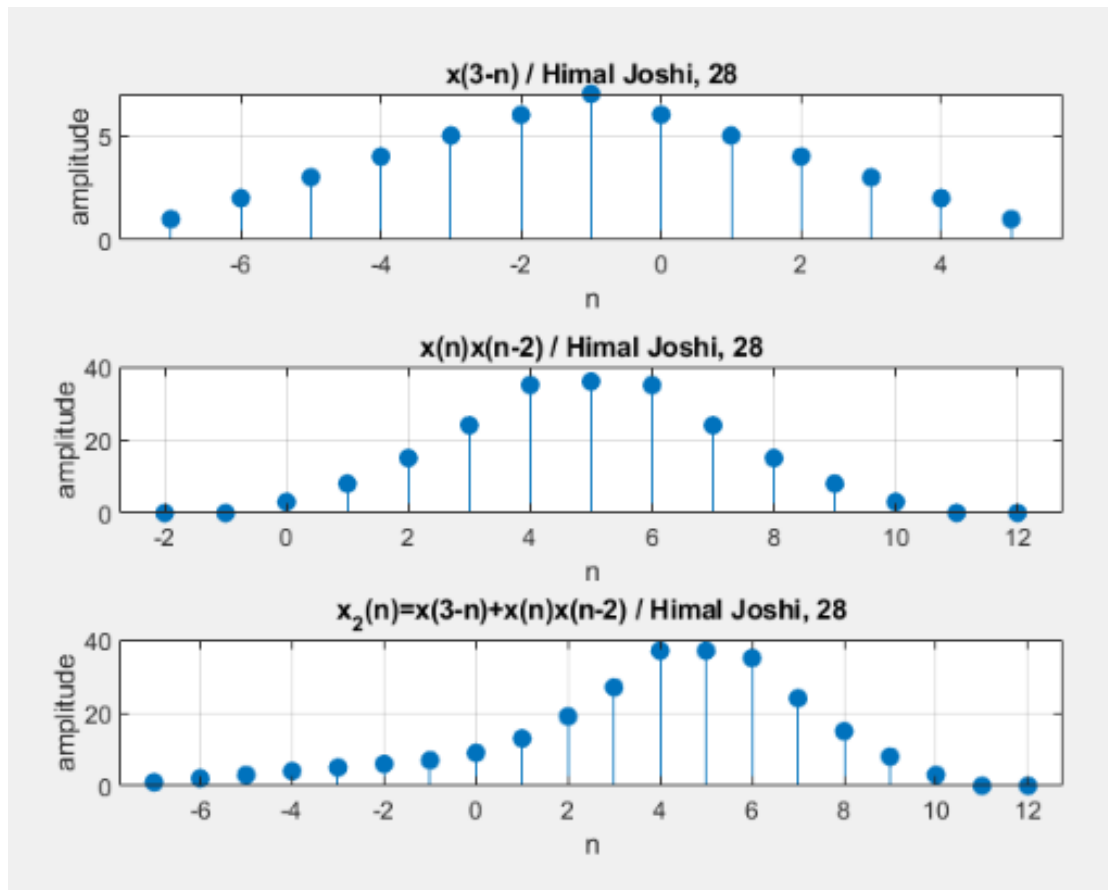


Figure 6: Folding and Multiplication

```
[u2,~] = stepseq(20,0,40);
[u3,~] = stepseq(40,0,40);

% Windowed signal
x = cos(0.1*pi*n).*(u1-u2) + ...
    0.5*sin(0.2*pi*n).*(u2-u3);

% Reference cosine
xref = cos(0.1*pi*n);

subplot(2,1,1)
stem(n,x,'filled');
xlabel('n');
ylabel('amplitude');
title('Windowed Sinusoidal Signal / Himal Joshi, 28');
grid on;

subplot(2,1,2)
```

```

stem(n,xref,'filled');
xlabel('n');
ylabel('amplitude');
title('Reference Signal cos(0.1\pin) / Himal Joshi, 28');
grid on;

```

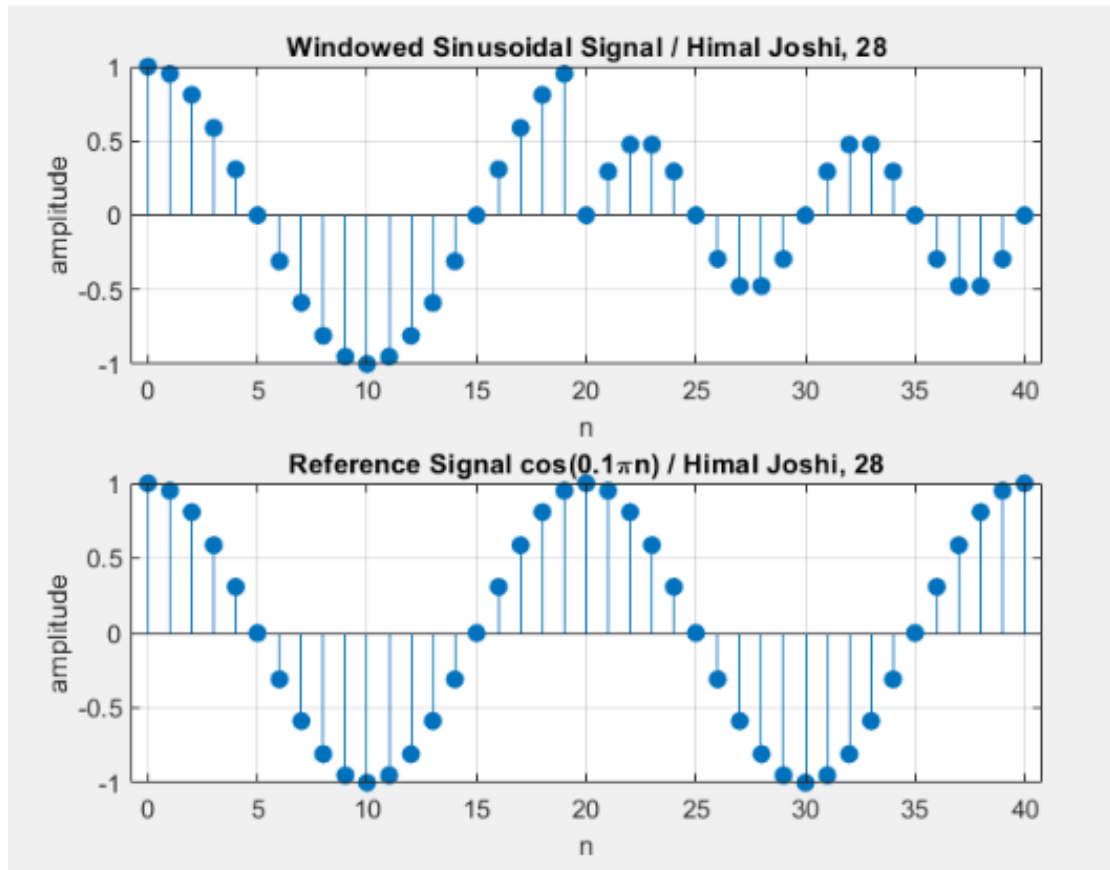


Figure 7: Windowed Sinusoidal Signal

Helper Function: energypower.m

```

function [E,P] = energypower(x)

% Computes energy and average power

E = sum(abs(x).^2);
P = E/length(x);

end

```

Task 8: Energy and Power Computation

```

clc;
close all;

```

```

clear all;

%% ===== Task 5 Signal =====

n = -2:10;
x = [1:7,6:-1:1];

[xs1,ns1] = sigshift(x,n,5);
xs1 = 2*xs1;

[xs2,ns2] = sigshift(x,n,-4);
xs2 = -3*xs2;

[x1,n1] = sigadd(xs1,ns1,xs2,ns2);

%% ===== Task 6 Signal =====

[xf,nf] = sigfold(x,n);
[xfs,nfs] = sigshift(xf,nf,3);

[xs,ns] = sigshift(x,n,2);

[xm,nm] = sigmult(x,n,xs,ns);

[x2,n2] = sigadd(xfs,nfs,xm,nm);

%% ===== Task 7 Signal =====

n3 = 0:40;

[u1,~] = stepseq(0,0,40);
[u2,~] = stepseq(20,0,40);
[u3,~] = stepseq(40,0,40);

x3 = cos(0.1*pi*n3).*(u1-u2) + ...
      0.5*sin(0.2*pi*n3).*(u2-u3);

%% ===== Energy & Power =====

```



```

[E1,P1] = energypower(x1);
[E2,P2] = energypower(x2);
[E3,P3] = energypower(x3);

fprintf('\n');
fprintf('-----\n');
fprintf(' Signal\t\tEnergy\t\tPower\n');
fprintf('-----\n');
fprintf(' Task 5\t\t%.2f\t\t%.2f\n',E1,P1);
fprintf(' Task 6\t\t%.2f\t\t%.2f\n',E2,P2);
fprintf(' Task 7\t\t%.2f\t\t%.2f\n',E3,P3);
fprintf('-----\n');

```

Signal	Energy	Power
Task 5	2763.00	125.59
Task 6	6317.00	315.85
Task 7	12.50	0.30

Figure 8: Energy and Power Computation Results

Task 8 Bonus: Energy Contribution per Sample

```

clc;
close all;
clear all;

n = 0:40;

```


Discussion

In this experiment, discrete-time unit impulse and unit step signals were generated and analyzed using MATLAB. Various signal operations such as shifting, folding, addition, and multiplication were performed using user-defined functions. Composite signals were created by combining elementary sequences, allowing observation of how signal transformations affect waveform behavior.

The windowed sinusoidal signal demonstrated the effect of limiting a signal to specific intervals using step functions. Energy and average power calculations were also performed for different signals, providing quantitative measures of signal strength. The obtained results matched the theoretical expectations and verified the correctness of the implemented MATLAB functions.

Conclusion

The experiment successfully demonstrated the generation and manipulation of discrete-time signals in MATLAB. Unit impulse and unit step sequences were created, and fundamental signal operations including shifting, folding, addition, and multiplication were implemented using reusable functions. Complex signals were constructed from elementary sequences, and their energy and average power were calculated. Therefore, the experiment provided practical understanding of essential concepts used in Digital Signal Processing.